

Types of Kubernetes Services & Their Work

A Kubernetes Service is a stable networking layer that exposes a group of Pods under one fixed address, even though individual Pods can be created, deleted, or replaced at any time. It automatically finds healthy Pods, load balances traffic between them, and allows other applications (inside or outside the cluster, depending on the Service type) to connect to your application without needing to know the Pods' changing IP addresses.

ClusterIP (Default Service)

Created by default when you run:

```
kubectl expose deployment mydep1 --port=80
```

- Provides a stable internal IP inside the cluster
- Connects and load balances traffic to pods
- Only accessible inside the Kubernetes cluster (not from outside)

For communication between microservices inside the cluster.

NodePort Service

```
kubectl expose deployment mydep1 --type=NodePort --port=80
```

- Opens a port on every worker node
- Allows access from outside the cluster using:

<Node-IP>:<NodePort>

Example:

192.168.1.6:30080

Good for testing or learning, but not used much in production.

LoadBalancer Service

```
kubectl expose deployment mydep1 --type=LoadBalancer --port=80
```

- Creates an external cloud load balancer
- Gives you a public IP or DNS name
- Automatically routes traffic to your pods

Used in real production environments (AWS, Azure, GCP, etc.).

ExternalName Service

- Does not connect to pods
- Maps a Kubernetes service name to an external domain

Example: yaml file

```
apiVersion: v1
kind: Service
metadata:
name: my-external-service
spec:
type: ExternalName
externalName: google.com
```

Example:

[my-external-service](#) → [google.com](#)

When your app in Kubernetes needs to call an external website or API.

Short Summary Table

Service Type	Main Work
ClusterIP	Internal access + load balancing
NodePort	Access via Node IP + Port
LoadBalancer	Public access via cloud LB
ExternalName	Map service to external domain

Main Difference: Where you can access them from

Feature	ClusterIP	LoadBalancer
Who can access it?	Only inside Kubernetes cluster	From outside the cluster (Internet)
IP Address	Private internal IP	Public external IP
Needs cloud provider?	No	Yes (AWS, Azure, GCP, etc.)
Default type?	Yes	No

Same thing they BOTH do

Both ClusterIP and LoadBalancer:

- Give a stable IP
- Load balance traffic across your pods
- Send traffic to the correct pod port

So their internal work is the same.

Create a Pod & Try to Delete and Check IP

```
controlplane:~$ kubectl create deployment mydep1 --image=docker.io/httpd --replicas=3
deployment.apps/mydep1 created
controlplane:~$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
mydep1-df8d87cf9-899cx	1/1	Running	0	112s	192.168.1.4	node01	<none>	<none>
mydep1-df8d87cf9-8zjkh	1/1	Running	0	112s	192.168.1.6	node01	<none>	<none>
mydep1-df8d87cf9-qjhc7	1/1	Running	0	112s	192.168.1.5	node01	<none>	<none>

```
controlplane:~$ kubectl delete pod mydep1-df8d87cf9-899cx
pod "mydep1-df8d87cf9-899cx" deleted from default namespace
controlplane:~$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
mydep1-df8d87cf9-2pqg5	1/1	Running	0	56s	192.168.1.7	node01	<none>	<none>
mydep1-df8d87cf9-8zjkh	1/1	Running	0	3m15s	192.168.1.6	node01	<none>	<none>
mydep1-df8d87cf9-qjhc7	1/1	Running	0	3m15s	192.168.1.5	node01	<none>	<none>

IP is changed after a delete

Create a Default Service (ClusterIP)

```
controlplane:~$ kubectl expose deployment mydep1 --port=80
service/mydep1 exposed
controlplane:~$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	12d
mydep1	ClusterIP	10.100.227.3	<none>	80/TCP	18s

```
controlplane:~$ kubectl describe svc mydep1
```

Name: mydep1
Namespace: default
Labels: app=mydep1
Annotations: <none>
Selector: app=mydep1
Type: ClusterIP
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.100.227.3
IPs: 10.100.227.3
Port: <unset> 80/TCP
TargetPort: 80/TCP
Endpoints: 192.168.1.6:80,192.168.1.7:80,192.168.1.5:80
Session Affinity: None
Internal Traffic Policy: Cluster
Events: <none>

```
controlplane:~$
```

IP: 10.100.227.3

This is the Service's permanent IP.

Even if pods die and new ones are created, this IP will not change.

The Service mydep1 gives a stable IP (10.100.227.3) and load balances traffic on port 80 across a three pods: 192.168.1.5, .6, and .7.

Distributes traffic across replicas.

```
controlplane:~$ kubectl exec -it mydep1-df8d87cf9-2pqg5 -- bash
root@mydep1-df8d87cf9-2pqg5:/usr/local/apache2# echo "Hii am Replica 1" > htdocs/
index.html
root@mydep1-df8d87cf9-2pqg5:/usr/local/apache2# exit
exit
controlplane:~$ kubectl exec -it mydep1-df8d87cf9-8zjkh -- bash
root@mydep1-df8d87cf9-8zjkh:/usr/local/apache2# echo "Hii am Replica 2" > htdocs/index.html
root@mydep1-df8d87cf9-8zjkh:/usr/local/apache2# exit
exit
controlplane:~$ kubectl exec -it mydep1-df8d87cf9-qjhc7 -- bash
root@mydep1-df8d87cf9-qjhc7:/usr/local/apache2# echo "Hii am Replica 3" > htdocs/
index.html
root@mydep1-df8d87cf9-qjhc7:/usr/local/apache2# █
```

The Service is load balancing traffic across all replicas.

```
controlplane:~$ curl 10.100.227.3
Hii am Replica 1
controlplane:~$ curl 10.100.227.3
Hii am Replica 2
controlplane:~$ curl 10.100.227.3
Hii am Replica 3
controlplane:~$ curl 10.100.227.3
Hii am Replica 1
controlplane:~$ curl 10.100.227.3
Hii am Replica 1
controlplane:~$ curl 10.100.227.3
Hii am Replica 2
controlplane:~$ curl 10.100.227.3
Hii am Replica 2
controlplane:~$ curl 10.100.227.3
Hii am Replica 2
controlplane:~$ █
```

Conclusion:

In this task, am learned that a Kubernetes Service gives our application one stable address even when Pods keep changing. It automatically connects to healthy Pods and shares (load balances) traffic between them, making our application more reliable and always reachable inside the cluster.